Lesson 5 Sidebars

Enemy Ranks

VIDEO GAME CREATION

These aren't part of the lesson's required work. Think of them the way a textbook sidebar works: a deeper, tangential look at something this lesson's build actually ran into - why one approach was chosen over the others, what the alternatives were, or an idea worth a longer look than the Handout has room for. Pull one out for a discussion moment if it fits, hand it to a curious student, or skip it entirely - nothing here is required reading.

How Are Colors Defined?

This lesson had every engine alternate two colors on a spawned enemy - crimson red, dusty pink, c_yellow, Make Linear Color, "200;25;55". Different letters and numbers for what's supposedly the same idea. Worth actually taking apart what a color IS to a computer, and why "set the color" and "blend the color" turn out to be two completely different operations.

Underneath every engine's different-looking color code, there are really only two families of answer to "what is a color," and this course's 8 engines lean on both. Get comfortable with both, and every engine's color API stops looking like a pile of unrelated syntax and starts looking like the same small idea, spelled differently.

RGB: How the Screen Actually Sees It

A screen doesn't paint anything - it's a grid of tiny lights, and every one of those lights is really three lights in one: a red one, a green one, and a blue one, layered on top of each other. RGB describes a color exactly the way the screen produces it: three numbers, one per light, each saying how bright that light is - off at the bottom, full brightness at the top, nothing conceptually fancier than a dimmer switch. Because it's genuinely combining LIGHT, not pigment, RGB mixing is additive: red light plus green light plus blue light, all the way up, makes white - the opposite of what happens when paints mix, and the first thing that trips people up about it.

Two scales show up across this course's engines for the exact same idea. The 8-bit scale runs each channel 0-255 (256 possible brightness steps per light) - GameMaker's make_color_rgb(), a hex code like #C81937, and Unity's Color32 all use this scale. The normalized float scale instead runs each channel 0-1, where 1.0 means the same "full brightness" that 255 means on the other scale - Godot's Color(r, g, b), Unity's plain Color(r, g, b), and Unreal's Linear Color all use this one. They're the same information, just counted differently: 200 out of 255 and 0.8 out of 1.0 are the same brightness, roughly 78%.

HSV/HSL: How a Person Actually Thinks About It

RGB is exactly how a screen makes color, and exactly the wrong shape for how a person thinks about color. "Make it a little darker" or "make it more washed-out" doesn't map onto changing just one of R, G, or B - usually all three numbers have to change together just to keep the same hue while adjusting how dark or vivid it looks. HSV describes the same color space along three axes that actually match how people talk about color: Hue is where it sits on the color wheel (one number from 0 to 360 degrees - 0/360 is red, 120 is green, 240 is blue, and everything between is a smooth rainbow); Saturation is how vivid or washed-out it is, from a pure, intense color down to gray; Value is how bright or dark it is, from the full color down to black. (HSL is a close cousin - same Hue, same idea, but its third axis, Lightness, runs from black up through the full color and on to white, rather than stopping at the full color the way Value does.)

This course's students have actually already met a real HSV interface without necessarily knowing to call it that: Scratch's Pen extension exposes "set pen color to ()" plus separate saturation and brightness blocks (from the same dropdown menu block) - genuine Hue, Saturation, and Value controls, not a disguised RGB picker. Worth flagging one Scratch-specific quirk directly: its Hue block runs 0-100, not the more common 0-360 - a real, documented Scratch choice, not a mistake to correct.

Named Shortcuts, Engine by Engine

Every engine also offers some kind of shortcut past typing raw numbers for common colors - but the shortcuts differ more than they first appear to, and it's worth being precise about which is which rather than assuming they're all the same trick:

- GameMaker: a fixed set of built-in global constants - c_red, c_yellow, c_white, c_black, c_blue, and more - each just a pre-made color value with a readable name. For anything outside that list, make_color_rgb(r, g, b) takes 0-255 per channel, and make_color_hsv(h, s, v) exists too - with a real gotcha worth flagging: GameMaker's HSV hue also runs 0-255, not the standard 0-360, so a straight mental conversion from "180 degrees" won't land where expected.
- Godot: two genuinely different shortcuts exist side by side. Color.YELLOW (and friends) are named constants, same idea as GameMaker's c_yellow. Separately, Color("yellow") - or Color("#ffff00") - is a real, documented constructor that parses a color name or a hex string directly into a Color value. Color.from_hsv(h, s, v) builds one from HSV instead of RGB.
- Unity: named colors are static properties, not a constructor - Color.yellow, Color.red, and friends. There is no Color("yellow") string constructor in Unity's C# API the way there is in Godot - a genuinely easy mix-up between the two engines, worth calling out directly since it's exactly the kind of "looks like it should exist" assumption this course tries to catch before it ships. Unity's actual hex-parsing tool is ColorUtility.TryParseHtmlString("#RRGGBB", out color), and Color.HSVToRGB(h, s, v) converts from HSV.
- GDevelop: no named-color shortcuts at all - its color actions take a plain "R;G;B" string (this lesson's Tint color = "200;25;55" is exactly this), always 0-255 per channel.
- Unreal: the Details panel's color picker shows named swatches purely as an editor convenience - under the hood, a color used for a material parameter is always a Linear Color struct (0-1 floats). Blueprint builds one from scratch with Make Linear Color - not Make Color, which builds a different, byte-based FColor struct (0-255) meant for other uses, a real distinction worth double-checking rather than assuming either node is

interchangeable with the other. C++ has genuine named constants (FLinearColor::Yellow, FLinearColor::Red) for a student who progresses that far.

- TIC-80: a genuinely different system, not a shortcut on top of RGB/HSV at all. TIC-80 draws from a small indexed palette - 16 total colors, each referenced everywhere by a plain number (0-15), not a color value directly. The palette itself (editable in the Sprite Editor, or rewritable at runtime) is what defines which actual RGB color a given index currently points to.
- 001 Game Creator: a hex color picker in Graphics Options (Actor > Graphics Options > Tint) - same underlying RGB idea as everywhere else, just entered as a hex triplet through a color-swatch UI rather than typed or wired as separate numbers.

Set vs. Blend: Two Different Operations Wearing the Same Word

Every one of this lesson's engines has some property that LOOKS like it directly sets an object's color - image_blend, modulate, SpriteRenderer.color, Tint color, Set Vector Parameter Value. None of them do that. Every one of them is actually a blend (also called a tint, or a multiply): result = original color x tint color, calculated separately per channel. That's a genuinely different operation from painting over a pixel with a fresh color, and the difference matters.

The technical version, worked through with real numbers: colors here are 0-1 floats per channel. A pixel that starts out pure red is (1, 0, 0). Tint it with this lesson's dusty rose/pink, (0.95, 0.4, 0.5), and the multiply runs channel by channel: R = 1 x 0.95 = 0.95, G = 0 x 0.4 = 0, B = 0 x 0.5 = 0 - the pixel comes out (0.95, 0, 0), a very slightly dimmed red, NOT the pink that was passed in. The G and B channels of the tint never had anything to work with, because the original pixel's G and B were already zero. Two edge cases worth knowing on sight: multiplying by white, (1, 1, 1), is the identity operation - 1 times anything leaves it completely unchanged, which is exactly why Lesson 13's white hit-flash silently did nothing on three engines' colored enemy sprite. Multiplying by black, (0, 0, 0), erases everything to black no matter what was there before, since anything times zero is zero.

The paint-mixing intuition gets the headline result right - a blend can only pull a color toward the tint and darken it, never brighten past what was already there, the same one-way limit real paint mixing has (you can't stir two paints together and get something lighter than the lightest one you started with). But two full pots of opaque paint stirred together isn't quite the mechanism - that's a subtractive pigment process (each pigment absorbs certain wavelengths of light, and mixing pigments compounds what gets absorbed), and it tends to produce a genuinely murkier, less predictable result than a clean per-channel multiply does. The closer real-world match for what image_blend/modulate/Tint color are actually doing is a transparent glaze or wash - a thin, see-through layer of color brushed over a painting that's already dry. The original brushstrokes still show through underneath, tinted by whatever color the glaze itself is. A clear or white glaze leaves the painting essentially untouched (the same reason a white tint is a no-op); a colored glaze can only pull what's underneath toward its own hue and darken it, never add a color, or brighten one, past what was already on the canvas.

Worth being honest about this lesson's own striped-row idea in that light: enemy_invader_red.png is not white or gray source art, so tinting it toward crimson or rose is still a genuine two-color multiply, not a clean color replacement - the same underlying situation as Lesson 2's muddy Recolor problem and Lesson 13's invisible white flash. It reads fine here specifically because the source art and both chosen tints are all already in the same red family - the multiply stays inside a plausible range of reds rather than drifting somewhere unexpected. Tint toward a saturated blue or green instead, and the same muddy result Lesson 2 already documented would show up again. Same rule, third lesson, third different-looking symptom -

which is exactly the point: understanding the rule once means being able to predict every place it's going to bite, instead of rediscovering it lesson by lesson.

- **The practical takeaway:** if the goal is a specific, exact target color - not just "noticeably different" - a blend/tint is the wrong tool no matter how carefully the numbers are chosen. That calls for an actual SET: swap in a different, pre-made sprite (this course's standard fix, covered in Lesson 2's Sidebar), or, for TIC-80 specifically, repoint the palette index itself rather than blending anything. A blend is exactly the right tool for "shift this toward a hue, however imprecisely" - which is genuinely all this lesson's striped-row idea ever needed.

Script vs. Visual Coding: Who Gets to Hold an Instance Reference?

Every engine's code path can spawn something, keep a handle to that one specific copy, and act on just it - that's just a local variable holding a reference. This lesson's two advanced-track engines answer "how do I expose that visually?" in genuinely different ways, and the GameMaker fix above is a direct result of which answer it picked.

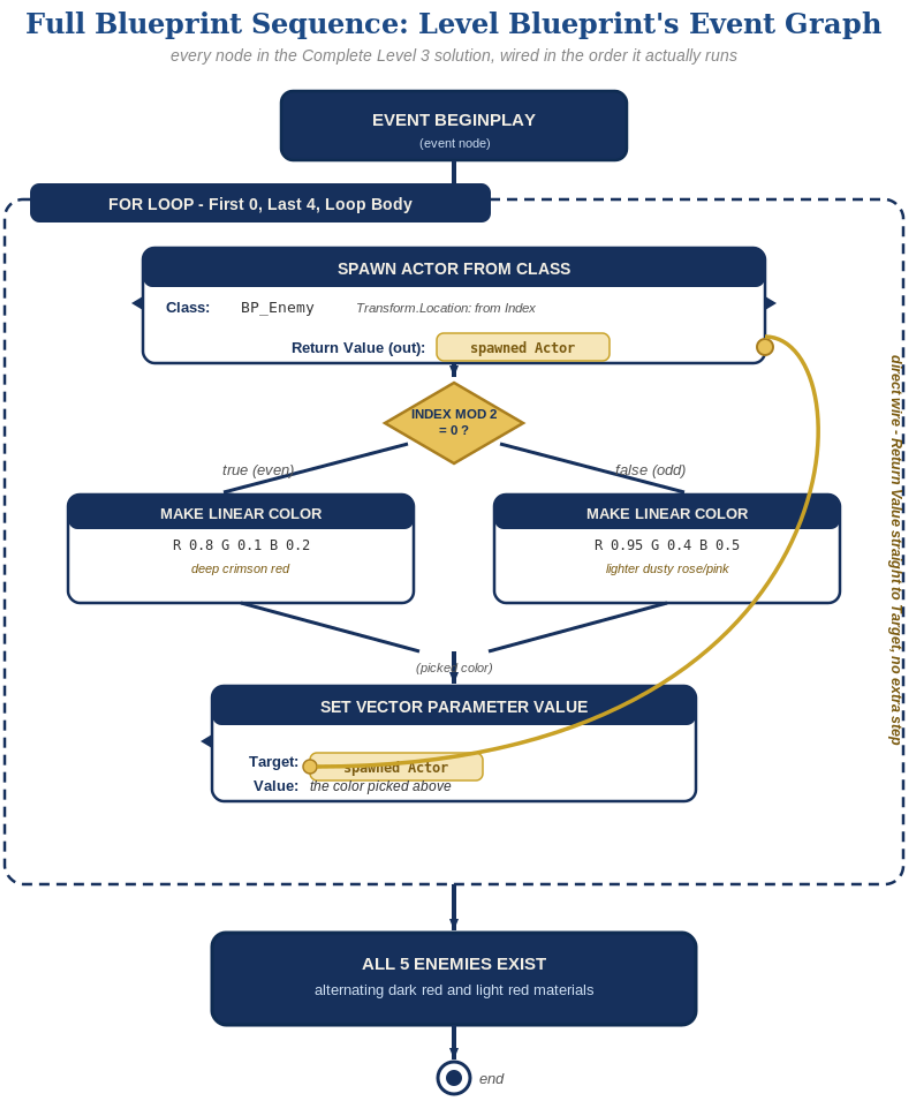
Both Unreal and GameMaker's visual layers need to solve the identical underlying problem this lesson's striped-row idea runs into: spawn an instance, then act on that one instance and no other, without re-finding it later. They land in different places, and the difference isn't a bug in either one - it's a real design choice about how much of "a reference to one specific object" a purely visual language should expose.

Unreal Blueprint: An Instance Reference Is a First-Class Wire

In C++, this is unremarkable - just a local variable holding a pointer, then a function call through it:

```
AEnemy* NewEnemy = GetWorld()->SpawnActor<AEnemy>(EnemyClass,
SpawnTransform);
NewEnemy->SetStripeColor(Index % 2 == 0 ? DarkRed : LightRed);
```

NewEnemy holds a reference to the one Actor SpawnActor just created; calling SetStripeColor through it runs that function on that Actor alone. Blueprint maps this almost directly, because it treats "a reference to one specific instance" as a real, first-class value - not a special case. Spawn Actor from Class has a Return Value output pin, and that pin's wire genuinely IS an Object Reference, the same kind of value as any other pin, just typed to one specific Actor. Drag it straight into the Target input of the very next node (the SetStripeColor call) and the call runs on exactly the instance the wire is carrying. Need to reuse that reference further down the graph? Right-click the pin and Promote to Variable - Blueprint creates a genuine "Enemy Object Reference" variable that can be dragged into any Target input anywhere in the graph, exactly like the C++ pointer.



This is the node-for-node equivalent of the Complete Level 3 solution's Blueprint description above - Spawn Actor's Return Value pin feeds Set Vector Parameter Value directly, the same way Unity's Instantiate output does.

Spawn Actor from Class's Return Value pin, wired straight into Set Vector Parameter Value's Target - the wire itself is the instance reference, no Promote to Variable step needed for a single, immediate use.

GameMaker GML Visual: A Scope Block Instead of a Socket

GameMaker's visual layer draws the line in a different place. Its purpose-built property blocks - Set Instance Variable, Set Instance Colour, Set Sprite, and similar - don't have an arbitrary Target input socket at all; instead they offer a fixed Action Scope dropdown in the block's own corner, and at first glance that dropdown only lists Self, Other, or a specific Object asset (which reaches every existing instance of that object, not one). There's no wire to drag a captured instance reference into, the way Blueprint's Target pin accepts one - so it looks, briefly, like GML Visual never built a way to reach one dynamically-created instance at all. It has one; it's just a block, not a socket.

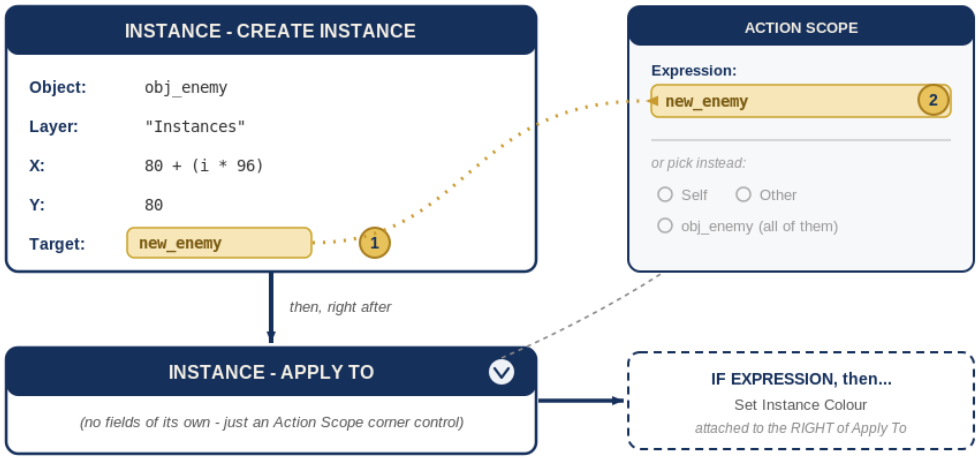
GameMaker's Answer: Apply To Re-Points What "Self" Means

The fix Robert found by hand, entirely within the Visual editor, is an "Instance - Apply To" block. Create Instance's own Target field captures the newly-made instance into a local variable (new_enemy) - the same first step as before. Apply To goes right after it, and whatever needs to act on that one instance - an If/Else block, in this lesson's case - attaches to the RIGHT of Apply To, not below it, which is what nests it inside Apply To's scope instead of just sequencing after it. Apply To's own Action Scope pop-up has an Expression field at the top, separate from its Self/Other/object list - type new_enemy into it, and "Self" now means that one captured instance for every block nested inside, including ordinary purpose-built ones like Set Instance Colour.

That makes GML Visual's answer closer to Unreal's than it first looks: both ultimately need the same value - the reference Create Instance/Spawn Actor from Class just captured - to reach the block that acts on it. Unreal carries that value on a wire, into a Target pin any node can expose. GameMaker carries it through a scope-setting block instead: Apply To's Expression field is the equivalent handoff point, just reached through a pop-up field and a rightward attachment rather than a draggable connection on the canvas. Once Apply To is in place, every block nested inside it is fully ordinary - no generic block, no typed workaround, nothing to reach for beyond the purpose-built one already being used everywhere else in the lesson.

GML Visual Wireframe: Capturing One Spawned Instance

inside the Repeat block, obj_spawner's Create event - Create Instance's Target, handed off to Apply To's Action Scope



1. Target is the only place Create Instance lets you capture a reference to the exact instance it just made.
2. Apply To's Action Scope Expression field is where that captured name goes instead of Self/Other/an object - everything attached to the RIGHT of Apply To (not below it) then runs with "Self" meaning that one instance, so the purpose-built Set Instance Colour block works normally - no generic block or typed workaround needed.

Apply To as the handoff point: Create Instance's Target field captures the new instance, and that same name goes straight into Apply To's Action Scope Expression field - a real, block-only connection, just not a wire on the canvas the way Unreal's Target pin is.

- **Why this is worth noticing:** Unreal and GameMaker solve the identical problem - reach one specific, dynamically-created instance and act on just it - with genuinely different visual mechanisms, even though both end up letting a purpose-built block do the actual work. Unreal treats an instance reference as a full citizen of its visual language, wireable anywhere a Target is expected. GameMaker reaches the same result through a

dedicated scope-setting block instead of a socket on every other block - which means a purpose-built block's own targeting dropdown looking limited isn't proof that the block can't be used; it's worth checking whether a scope block like Apply To is available before assuming a generic or typed fallback is the only option. Worth checking which mechanism a given visual engine has actually built before assuming its solution should look like another engine's.